تمرین کامپیوتری چهارم

مبانی بینایی کامپیوتر

بهار 1405

روجا آقاجانی 810101380

مانا مرادی 810101517

سوال 1)

در این بخش، مسیر پوشه‌های train و test در کد مشخص شده و داده‌ها با استفاده از تابع image_dataset_from_directory بارگذاری شده‌اند. از آنجا که داده‌ها به‌صورت پوشه‌بندی‌شده هستند، نام هر پوشه به‌عنوان نام کلاس در نظر گرفته می‌شود.

برای بررسی صحت بارگذاری داده‌ها، شکل یک batch از تصاویر و برچسب‌ها چاپ شده است. در صورتی که اندازه‌ی batch برابر با ۳۲ باشد، شکل تصاویر به‌صورت (32, 150, 150, 3)و شکل برچسب‌ها به‌صورت (32,) خواهد بود. این خروجی نشان می‌دهد که در هر batch تعداد ۳۲ تصویر وجود دارد و برای هر تصویر یک برچسب عددی اختصاص داده شده است.

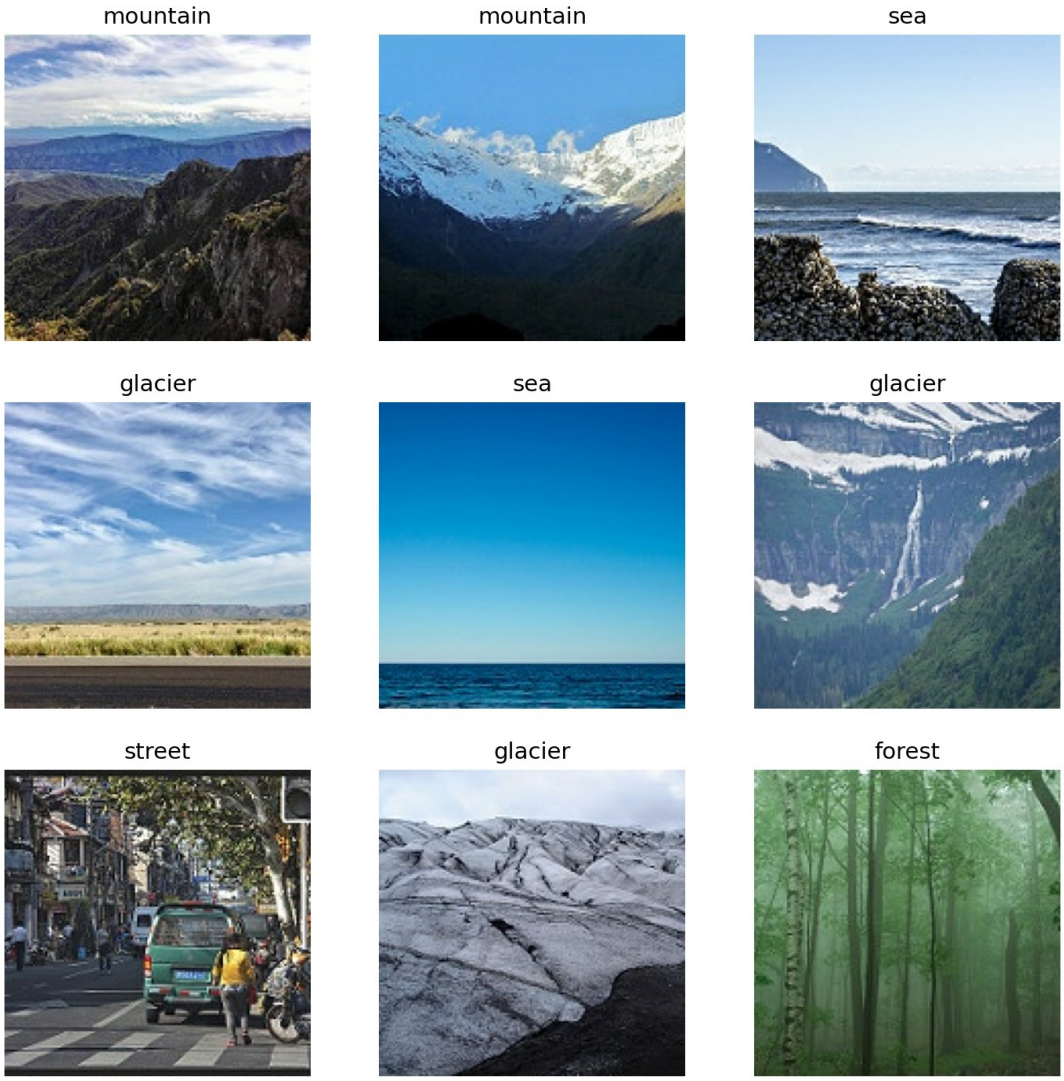
همچنین بخشی از داده‌های آموزشی به‌عنوان validation جدا شده است. در اینجا، ۲۰ درصد از داده‌های train برای validationو ۸۰ درصد باقی‌مانده برای training استفاده می‌شود. داده‌های validation برای بررسی عملکرد مدل روی داده‌هایی به‌کار می‌روند که مستقیماً در فرآیند آموزش استفاده نشده‌اند.

در ادامه، چند نمونه تصویر همراه با برچسب آن‌ها نمایش داده شده است تا از درست بودن بارگذاری تصاویر و هماهنگی labelها با محتوای واقعی تصاویر اطمینان حاصل شود.

هر تصویر در این دیتاست دارای ابعاد $3 \times 150 \times 150$ است. دو عدد ۱۵۰ نشان‌دهنده‌ی ارتفاع و عرض تصویر هستند و عدد ۳ بیانگر سه کانال رنگی RGB است. بنابراین هر تصویر به‌صورت یک آرایه‌ی سه‌بعدی نمایش داده می‌شود.

در شبکه‌یCNN ، تصویر در ابتدای کار به‌صورت مستقیم Flatten نمی‌شود، زیرا ارتباط مکانی بین پیکسل‌ها برای تشخیص ویژگی‌هایی مانند لبه‌ها، بافت‌ها و شکل کلی تصویر اهمیت دارد. ابتدا لایه‌های کانولوشنی این ویژگی‌ها را استخراج می‌کنند. سپس خروجی چندبعدی بخش کانولوشنی، قبل از ورود به لایه‌هایDense ، با استفاده از لایه‌ی Flatten به یک بردار یک‌بعدی تبدیل می‌شود. بنابراین Flatten برای آماده‌سازی خروجی استخراج‌شده جهت طبقه‌بندی نهایی استفاده می‌شود.

در شکل زیر، چند نمونه تصویر از داده‌های آموزشی نمایش داده شده‌اند. عنوان بالای هر تصویر نشان‌دهنده‌ی برچسب یا کلاس واقعی آن تصویر است. این نمایش برای بررسی اولیه‌ی داده‌ها و اطمینان از درست بودن بارگذاری تصاویر و هماهنگی برچسب‌ها با محتوای تصاویر انجام شده است:




```
Class names:
['buildings', 'forest', 'glacier', 'mountain', 'sea', 'street']

Batch shapes:
Images batch shape: (32, 150, 150, 3)
Labels batch shape: (32,)
```

سوال 2)

داده‌افزایی یا همان augmentation باعث کاهش احتمال overfitting می‌شود، زیرا مدل در زمان آموزش فقط تصاویر اصلی و ثابت را مشاهده نمی‌کند، بلکه نسخه‌های کمی تغییریافته از همان تصاویر را نیز می‌بیند. این تغییرات می‌توانند شامل چرخش جزئی، برعکس‌شدن افقی، یا بزرگ‌نمایی و کوچک‌نمایی تصویر باشند.

وقتی مدل فقط تصاویر اصلی را ببیند، ممکن است جزئیات خاص همان تصاویر آموزشی را حفظ کند و به‌جای یادگیری ویژگی‌های کلی هر کلاس، بیش از حد به داده‌های train وابسته شود. در این حالت، دقت مدل روی داده‌های آموزشی بالا می‌رود، اما روی داده‌های جدید مثل validation یا test عملکرد ضعیف‌تری خواهد داشت.

با استفاده از augmentation ، تنوع داده‌های آموزشی بیشتر می‌شود و مدل مجبور می‌شود ویژگی‌های عمومی‌تر هر کلاس را یاد بگیرد. برای مثال، مدل باید یاد بگیرد که تصویر دریا حتی اگر کمی چرخیده یا بزرگ‌نمایی شده باشد، همچنان مربوط به کلاس دریا است. بنابراین augmentation باعث می‌شود مدل بهتر تعمیم پیدا کند و احتمال overfitting کاهش یابد.

در شکل زیر، یک تصویر آموزشی چند بار با تغییرات تصادفی مانند چرخش، برعکس‌شدن افقی یا zoom نمایش داده شده است. این خروجی نشان می‌دهد که داده‌افزایی روی تصویر اعمال شده است.



سوال 3)

در این بخش، یک مدل شبکه عصبی کانولوشنی برای طبقه‌بندی تصاویر رنگی با ابعاد $3 \times 150 \times 150$طراحی شد. خروجی model.summary() نشان می‌دهد که مدل شامل لایه‌های داده‌افزایی، نرمال‌سازی، چهار لایه‌ی کانولوشنی، چهار لایه‌یMaxPooling ، لایه‌یFlatten ، یک لایه‌ی Dense و لایه‌ی خروجی ۶ نورونی است. از آنجا که دیتاست شامل ۶ کلاس است، لایه‌ی خروجی نیز دارای ۶ نورون در نظر گرفته شده است.

در این شکل، ترتیب لایه‌های مدل، ابعاد خروجی هر لایه و تعداد پارامترهای هر بخش نمایش داده شده است.

در ابتدای مدل، لایه‌ی data_augmentation قرار گرفته است. این لایه روی تصاویر آموزشی تغییراتی مانند برعکس‌کردن افقی، چرخش جزئی و zoom تصادفی اعمال می‌کند. خروجی این لایه همچنان دارای ابعاد (150, 150, 3) است، زیرا این لایه فقط ظاهر تصویر را تغییر می‌دهد و ابعاد اصلی تصویر را ثابت نگه می‌دارد. تعداد پارامترهای این لایه صفر است، چون وزن یا bias قابل آموزش ندارد.

پس از آن، لایه‌ی Rescaling قرار دارد. این لایه مقدار پیکسل‌ها را از بازه‌ی ۰ تا ۲۵۵ به بازه‌ی ۰ تا ۱ تبدیل می‌کند. این کار باعث می‌شود ورودی شبکه از نظر عددی در مقیاس مناسب‌تری قرار گیرد. این لایه نیز پارامتر قابل آموزش ندارد، زیرا فقط یک تبدیل ثابت روی مقدار پیکسل‌ها انجام می‌دهد.

در ادامه، چهار لایه‌ی Conv2D استفاده شده است. این لایه‌ها وظیفه‌ی استخراج ویژگی‌های تصویری را بر عهده دارند. لایه‌های کانولوشنی با استفاده از فیلترها، ویژگی‌هایی مانند لبه‌ها، خطوط، بافت‌ها و الگوهای پیچیده‌تر را از تصویر استخراج می‌کنند. تعداد فیلترها در این مدل به‌ترتیب ۳۲، ۶۴، ۱۲۸ و ۱۲۸ است. افزایش تعداد فیلترها در لایه‌های عمیق‌تر باعث می‌شود مدل بتواند ویژگی‌های پیچیده‌تر و سطح بالاتری را یاد بگیرد.

اولین لایه‌ی کانولوشنی خروجی‌ای با ابعاد (148, 148, 32) تولید کرده است. ابعاد تصویر از ۱۵۰ به ۱۴۸ کاهش یافته، زیرا از فیلتر 3×3 بدون padding استفاده شده است. تعداد پارامترهای این لایه برابر با 896 است. این مقدار از تعداد وزن‌های فیلترها و bias ها به‌دست می‌آید:

$$(3 \times 3 \times 3 + 1) \times 32 = 896$$

در این رابطه، 3×3 اندازه‌ی فیلتر، عدد ۳ تعداد کانال‌های ورودیRGB ، عدد ۱ مربوط به bias هر فیلتر، و عدد ۳۲ تعداد فیلترهای لایه است.

بعد از هر لایه‌ی کانولوشنی، یک لایه‌ی MaxPooling2D قرار گرفته است. نقش این لایه کاهش ابعاد مکانی feature map است. برای مثال، خروجی اولین لایه‌ی کانولوشنی از (148, 148, 32) به (74, 74, 32) تبدیل شده است. این کاهش ابعاد باعث کم شدن حجم محاسبات و ساده‌تر شدن پردازش در لایه‌های بعدی می‌شود. لایه‌های MaxPooling پارامتر قابل آموزش ندارند، زیرا فقط از هر ناحیه مقدار بیشینه را انتخاب می‌کنند.

لایه‌ی دوم کانولوشنی خروجی‌ای با ابعاد (72, 72, 64) دارد و تعداد پارامترهای آن 18,496 است. در این مرحله، ورودی لایه ۳۲ کاناله است و لایه دارای ۶۴ فیلتر می‌باشد. بنابراین تعداد پارامترها به‌صورت زیر محاسبه می‌شود:

$$(3 \times 3 \times 32 + 1) \times 64 = 18{,}496$$

لایه‌ی سوم کانولوشنی خروجی‌ای با ابعاد (34, 34, 128) دارد و تعداد پارامترهای آن 73,856 است. در این لایه، ورودی ۶۴ کاناله است و ۱۲۸ فیلتر استفاده شده است:

$$(3 \times 3 \times 64 + 1) \times 128 = 73{,}856$$

لایه‌ی چهارم کانولوشنی خروجی‌ای با ابعاد (15, 15, 128) دارد و تعداد پارامترهای آن 147,584است. در این لایه، ورودی ۱۲۸ کاناله است و ۱۲۸ فیلتر وجود دارد:

$$(3 \times 3 \times 128 + 1) \times 128 = 147{,}584$$

پس از چهار مرحله کانولوشن و MaxPooling ، خروجی مدل به ابعاد (7, 7, 128)می‌رسد. این خروجی هنوز چندبعدی است و نمی‌تواند مستقیماً وارد لایه‌ی Dense شود. به همین دلیل، از لایه‌ی Flatten استفاده شده است. این لایه خروجی چندبعدی را به یک بردار یک‌بعدی تبدیل می‌کند. در این مدل، خروجی (7, 7, 128) به برداری با طول 6272تبدیل می‌شود:

$$7 \times 7 \times 128 = 6272$$

لایه‌ی Flatten نیز پارامتر قابل آموزش ندارد، چون فقط شکل داده را تغییر می‌دهد.

بعد ازFlatten ، یک لایه‌ی Dense با ۱۲۸ نورون قرار گرفته است. این لایه دارای 802,944 پارامتر است و بیشترین تعداد پارامترهای مدل مربوط به همین لایه است. دلیل زیاد بودن تعداد پارامترها این است که بردار ورودی آن طول ۶۲۷۲ دارد و هر ورودی به همه‌ی ۱۲۸ نورون متصل می‌شود:

$$6272 \times 128 + 128 = 802{,}944$$

عدد ۱۲۸ در انتهای رابطه مربوط بهbias های ۱۲۸ نورون است. نقش این لایه ترکیب ویژگی‌های استخراج‌شده توسط لایه‌های کانولوشنی و کمک به تصمیم‌گیری نهایی برای طبقه‌بندی تصویر است.

در انتهای مدل، یک لایه‌ی Dense با ۶ نورون قرار دارد. تعداد نورون‌های این لایه برابر با تعداد کلاس‌های دیتاست است. این لایه دارای 774پارامتر است:

$$128 \times 6 + 6 = 774$$

تابع فعال‌سازی این لایه Softmax است. Softmax خروجی مدل را به احتمال تعلق تصویر به هر یک از ۶ کلاس تبدیل می‌کند و کلاسی که بیشترین احتمال را داشته باشد، به‌عنوان پیش‌بینی نهایی انتخاب می‌شود.

در مجموع، تعداد کل پارامترهای مدل برابر با 1,044,550 است. همه‌ی این پارامترها قابل آموزش هستند و تعداد پارامترهای غیرقابل‌آموزش برابر با صفر است. لایه‌هایی مانند data_augmentation، Rescaling، MaxPooling2D و Flatten پارامتر قابل آموزش ندارند. پارامترهای قابل آموزش اصلی مربوط به لایه‌های Conv2D و Dense هستند، زیرا این لایه‌ها وزن و bias دارند و این مقادیر در طول آموزش به‌روزرسانی می‌شوند.

خروجی:

```
Found 14034 files belonging to 6 classes.
Using 2806 files for validation.
Found 3000 files belonging to 6 classes.
Model: "sequential"
```

Layer (type)	Output Shape	Param #
data_augmentation (Sequential)	(None, 150, 150, 3)	0
rescaling (Rescaling)	(None, 150, 150, 3)	0
conv2d (Conv2D)	(None, 148, 148, 32)	896
max_pooling2d (MaxPooling2D)	(None, 74, 74, 32)	0
conv2d_1 (Conv2D)	(None, 72, 72, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 36, 36, 64)	0
conv2d_2 (Conv2D)	(None, 34, 34, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 17, 17, 128)	0
conv2d_3 (Conv2D)	(None, 15, 15, 128)	147,584
max_pooling2d_3 (MaxPooling2D)	(None, 7, 7, 128)	0
flatten (Flatten)	(None, 6272)	0
dense (Dense)	(None, 128)	802,944
dense_1 (Dense)	(None, 6)	774

```
Total params: 1,044,550 (3.98 MB)
Trainable params: 1,044,550 (3.98 MB)
Non-trainable params: 0 (0.00 B)
```

سوال 4)

در این بخش، مدل با optimizer نوع Adam و تابع خطای SparseCategoricalCrossentropy کامپایل شد.
همچنین معیار ارزیابی مدل، accuracy در نظر گرفته میشود.

از آنجا که مسئله‌ی این تمرین یک مسئله‌ی طبقه‌بندی چندکلاسه است، استفاده از تابع خطای Crossentropy مناسب میباشد. در این مسئله، هر تصویر فقط به یکی از ۶ کلاس موجود تعلق دارد؛ بنابراین مدل باید برای هر تصویر، احتمال تعلق آن به هر یک از کلاس‌ها را محاسبه کند.

در لایه‌ی خروجی مدل از تابع فعال‌سازی Softmax استفاده شده است همانطور که در بخش سه ذکر شد Softmax خروجی مدل را به ۶ مقدار احتمالی تبدیل می‌کند؛ به‌طوری که هر مقدار نشان‌دهنده‌ی احتمال تعلق تصویر به یکی از کلاس‌ها است. تابع Crossentropy اختلاف بین احتمال پیش‌بینی‌شده توسط مدل و برچسب واقعی تصویر را اندازه‌گیری می‌کند. اگر مدل به کلاس درست احتمال بالایی بدهد، مقدار loss کمتر می‌شود و اگر به کلاس درست احتمال کمی بدهد، مقدار loss بیشتر می‌شود.

در اینجا از SparseCategoricalCrossentropy استفاده شده است، چون برچسب کلاس‌ها به‌صورت عددی هستند. یعنی هر تصویر با یک عدد مانند ۰ تا ۵ مشخص می‌شود و برچسب‌ها به‌صورت one-hot نمایش داده نشده‌اند. اگر برچسب‌ها به‌صورت one-hot بودند، باید از CategoricalCrossentropy استفاده می‌شد؛ اما اینجا به علت عددی بودن label ها، انتخاب SparseCategoricalCrossentropy مناسب میباشد.

سوال 5)

```
Non-trainable params: 0 (0.00 B)
Epoch 1/10

Epoch 1: val_accuracy improved from -inf to 0.66393, saving model to cnn_model_best_question_5.keras
1404/1404 - 391s - 278ms/step - accuracy: 0.5455 - loss: 1.1397 - val_accuracy: 0.6639 - val_loss: 0.8948 - learning_rate: 0.0010
Epoch 2/10

Epoch 2: val_accuracy did not improve from 0.66393
1404/1404 - 390s - 278ms/step - accuracy: 0.6661 - loss: 0.8763 - val_accuracy: 0.6497 - val_loss: 0.9329 - learning_rate: 0.0010
Epoch 3/10

Epoch 3: val_accuracy improved from 0.66393 to 0.77227, saving model to cnn_model_best_question_5.keras
1404/1404 - 387s - 276ms/step - accuracy: 0.7199 - loss: 0.7566 - val_accuracy: 0.7723 - val_loss: 0.6431 - learning_rate: 0.0010
Epoch 4/10

Epoch 4: val_accuracy did not improve from 0.77227
1404/1404 - 386s - 275ms/step - accuracy: 0.7504 - loss: 0.6850 - val_accuracy: 0.7384 - val_loss: 0.7205 - learning_rate: 0.0010
Epoch 5/10

Epoch 5: val_accuracy improved from 0.77227 to 0.79330, saving model to cnn_model_best_question_5.keras
1404/1404 - 421s - 300ms/step - accuracy: 0.7663 - loss: 0.6464 - val_accuracy: 0.7933 - val_loss: 0.5739 - learning_rate: 0.0010
Epoch 6/10

Epoch 6: val_accuracy did not improve from 0.79330
1404/1404 - 420s - 299ms/step - accuracy: 0.7798 - loss: 0.6084 - val_accuracy: 0.7805 - val_loss: 0.6016 - learning_rate: 0.0010
Epoch 7/10

Epoch 7: val_accuracy improved from 0.79330 to 0.80898, saving model to cnn_model_best_question_5.keras
1404/1404 - 414s - 295ms/step - accuracy: 0.7894 - loss: 0.5776 - val_accuracy: 0.8090 - val_loss: 0.5294 - learning_rate: 0.0010
Epoch 8/10

Epoch 8: val_accuracy did not improve from 0.80898
1404/1404 - 409s - 291ms/step - accuracy: 0.7967 - loss: 0.5564 - val_accuracy: 0.7780 - val_loss: 0.6170 - learning_rate: 0.0010
Epoch 9/10

Epoch 9: val_accuracy did not improve from 0.80898

Epoch 9: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
1404/1404 - 404s - 288ms/step - accuracy: 0.8052 - loss: 0.5467 - val_accuracy: 0.7933 - val_loss: 0.5847 - learning_rate: 0.0010
Epoch 10/10

Epoch 10: val_accuracy improved from 0.80898 to 0.81468, saving model to cnn_model_best_question_5.keras
1404/1404 - 405s - 288ms/step - accuracy: 0.8337 - loss: 0.4562 - val_accuracy: 0.8147 - val_loss: 0.5060 - learning_rate: 5.0000e-04
Training history:
   accuracy      loss  val_accuracy  val_loss  learning_rate
0  0.545511  1.139668      0.663934  0.894808         0.0010
1  0.666103  0.876258      0.649679  0.932934         0.0010
2  0.719897  0.756613      0.772274  0.643075         0.0010
3  0.750445  0.684997      0.738418  0.720492         0.0010
4  0.766299  0.646361      0.793300  0.573932         0.0010
5  0.779836  0.608392      0.780470  0.601583         0.0010
6  0.789366  0.577568      0.808981  0.529353         0.0010
7  0.796669  0.556382      0.777976  0.617040         0.0010
8  0.805219  0.546712      0.793300  0.584731         0.0010
9  0.833719  0.456179      0.814683  0.506042         0.0005
Training completed.
```

طبق تصویر بالا ، مقدار Training Accuracy از 0.5455 در epoch اول به 0.8337 در epoch دهم افزایش یافته است. همچنین مقدار Training Loss از 1.1397 به 0.4562 کاهش پیدا کرده است. این روند نشان می‌دهد که مدل در طول آموزش توانسته است ویژگی‌های داده‌های آموزشی را یاد بگیرد و خطای خود را کاهش دهد.

مقدار Validation Accuracy نیز در پایان آموزش به 0.8147رسیده است و مقدار Validation Loss برابر با 0.5060 شده است. این مقادیر نشان می‌دهند که مدل علاوه بر داده‌های آموزشی، روی داده‌های validation نیز عملکرد مناسبی داشته است.

همچنین مقدارهای loss و accuracy مربوط به هر epoch ذخیره شدند تا در بخش بعدی بتوان از آن‌ها استفاده کرد.

سوال 6)

در این بخش، مدل آموزش‌دیده روی داده‌های test ارزیابی شد. پس از اجرای مدل روی داده‌هایtest ، مقدارهای زیر به‌دست آمد:

```
Test loss:
0.5313757061958313
Test accuracy:
0.8186666369438171
The model reached the required minimum test accuracy.
```

دقت مدل روی داده‌های test برابر با حدود 81.87% است. از آنجا که حداقل دقت مورد قبول در صورت سؤال برابر با 80% در نظر گرفته شده است، مدل توانسته است شرط دقت مورد نیاز را برآورده کند.

در بخش بعد، چند نمونه از تصاویر test نمایش داده شدند و برای هر تصویر، برچسب واقعی و برچسب پیش‌بینی‌شده توسط مدل مشخص شد. در شکل زیر، مقدار True نشان‌دهنده‌ی برچسب واقعی تصویر و مقدار Pred نشان‌دهنده‌ی برچسب پیش‌بینی‌شده توسط مدل است. اگر این دو مقدار یکسان باشند، مدل تصویر را درست طبقه‌بندی کرده است و اگر متفاوت باشند، پیش‌بینی مدل اشتباه بوده است.



گزارش طبقه‌بندی مدل روی داده‌های test به‌صورت زیر به‌دست آمد:

```
Classification report:
              precision    recall  f1-score   support

   buildings       0.70      0.91      0.79       437
      forest       0.93      0.97      0.95       474
     glacier       0.83      0.72      0.77       553
    mountain       0.73      0.82      0.77       525
         sea       0.86      0.74      0.80       510
      street       0.92      0.78      0.84       501

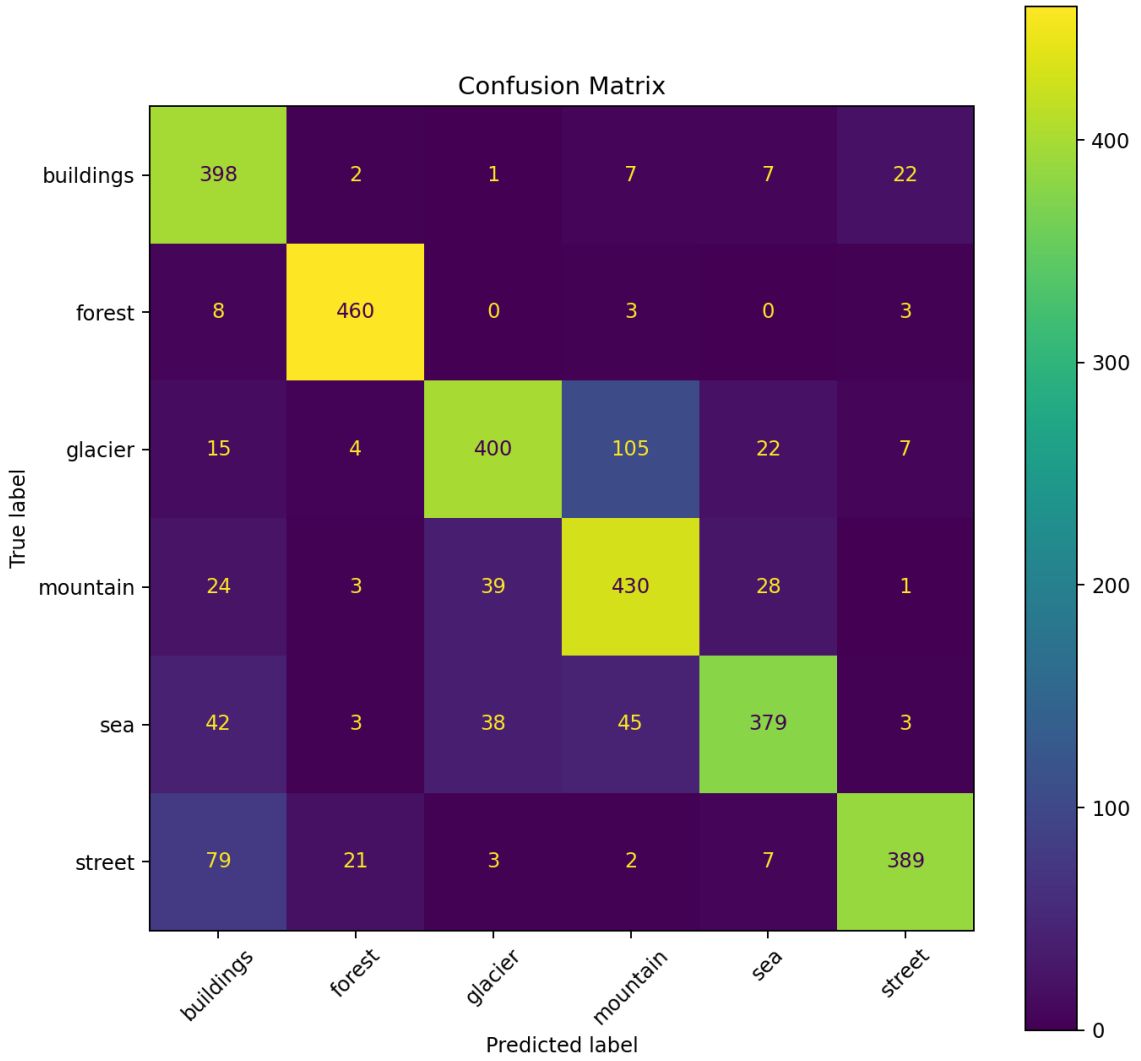
    accuracy                           0.82      3000
   macro avg       0.83      0.82      0.82      3000
weighted avg       0.83      0.82      0.82      3000
```

بر اساس تصویر بالا، بهترین عملکرد مدل مربوط به کلاس forest است؛ زیرا مقدارهای precision = 0.93 ، recall = 0.97و f1-score = 0.95برای این کلاس به‌دست آمده‌اند. این یعنی مدل تصاویر جنگل را با دقت بالایی تشخیص داده است.

کلاس buildings دارای recall = 0.91 است، یعنی بیشتر تصاویر واقعی ساختمان به‌درستی تشخیص داده شده‌اند. اما precision = 0.70 برای این کلاس نشان می‌دهد که تعدادی از تصاویری که مدل به‌عنوان ساختمان پیش‌بینی کرده، واقعاً متعلق به کلاس‌های دیگر بوده‌اند.

برای کلاس glacier مقدار recall = 0.72 و برای کلاس sea مقدار recall = 0.74 به‌دست آمده است. این مقادیر نشان می‌دهند که مدل بخشی از تصاویر واقعی این کلاس‌ها را با کلاس‌های دیگر اشتباه گرفته است.

در شکل زیر، ماتریس درهم‌ریختگی مدل روی داده‌های test نمایش داده می‌شود. مقدارهای روی قطر اصلی نشان‌دهنده‌ی پیش‌بینی‌های درست هستند و مقدارهای خارج از قطر اصلی نشان‌دهنده‌ی خطاهای مدل می‌باشند.



با توجه به خروجی خطاها، بیشترین اشتباهات مدل مربوط به موارد زیر است:

```
Number of wrong predictions:
544
Some wrong predictions:
True: buildings  | Predicted: sea
True: buildings  | Predicted: street
True: buildings  | Predicted: sea
True: buildings  | Predicted: mountain
True: buildings  | Predicted: forest
True: buildings  | Predicted: street
True: buildings  | Predicted: street
True: buildings  | Predicted: street
True: buildings  | Predicted: mountain
True: buildings  | Predicted: street
Most frequent confusion pairs:
glacier was predicted as mountain : 105
street was predicted as buildings : 79
sea was predicted as mountain : 45
sea was predicted as buildings : 42
mountain was predicted as glacier : 39
sea was predicted as glacier : 38
mountain was predicted as sea : 28
mountain was predicted as buildings : 24
buildings was predicted as street : 22
glacier was predicted as sea : 22
```

در مجموع، تعداد پیش‌بینی‌های اشتباه مدل برابر با 544 نمونه از کل تصاویر test بوده است. بنابراین مدل بیشتر تصاویر را درست طبقه‌بندی کرده، اما هنوز بین بعضی کلاس‌ها خطا دارد.

بیشترین خطا مربوط به این است که تصاویر واقعی کلاس glacier به‌اشتباه به‌عنوان mountain پیش‌بینی شده‌اند. این خطا 105 بار رخ داده است. دلیل این اشتباه می‌تواند شباهت ظاهری بین کوه و یخچال باشد؛ زیرا هر دو کلاس ممکن است شامل برف، سنگ، آسمان، رنگ‌های روشن و ساختارهای طبیعی مشابه باشند.

خطای پرتکرار بعدی مربوط به این است که تصاویر واقعی street به‌عنوان buildings پیش‌بینی شده‌اند. این خطا 79بار رخ داده است. این اشتباه قابل انتظار است، زیرا بسیاری از تصاویر خیابان شامل ساختمان‌ها هستند و مدل ممکن است به‌جای تمرکز روی کل صحنه‌ی خیابان، بیشتر به ساختمان‌های موجود در تصویر توجه کند.

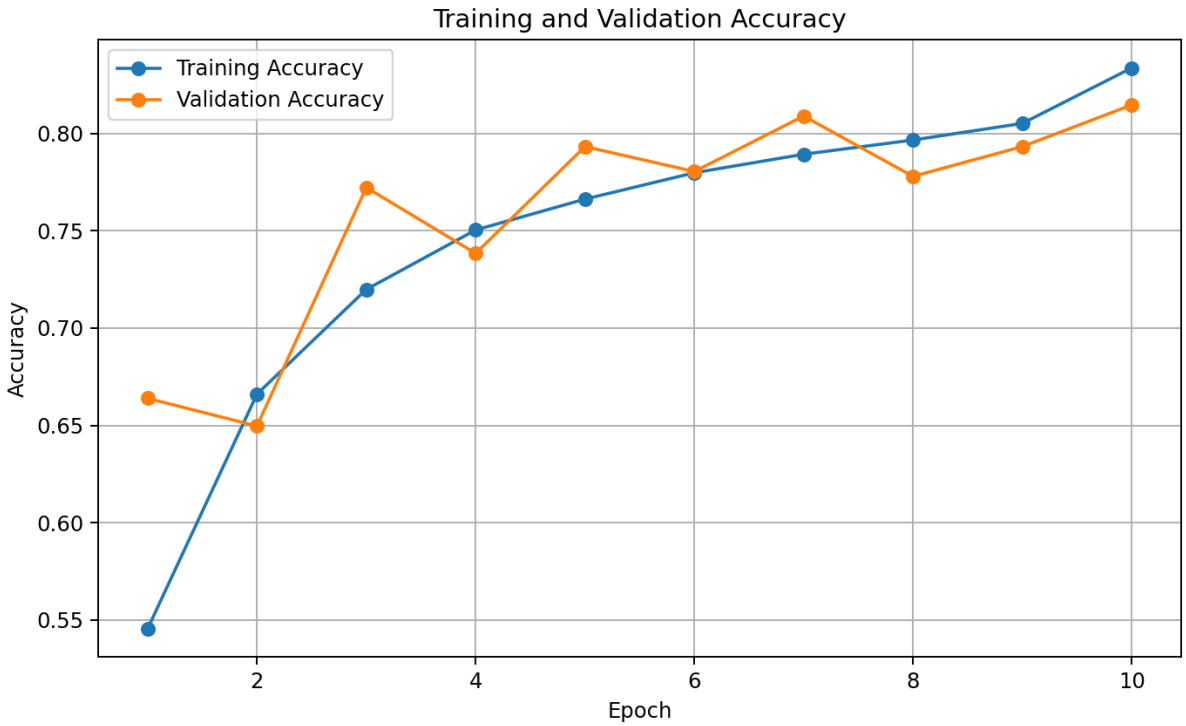
همچنین مدل در برخی موارد تصاویر sea را به‌عنوان mountain، buildings یا glacier پیش‌بینی کرده است. این نشان می‌دهد که بعضی تصاویر دریا احتمالاً شامل آسمان، ساحل، سنگ، کوه یا نواحی روشن بوده‌اند و از نظر بصری با چند کلاس دیگر شباهت داشته‌اند.

به‌طور کلی، خطاهای مدل بیشتر بین کلاس‌هایی رخ داده‌اند که از نظر ظاهری شباهت بیشتری به یکدیگر دارند؛ مانند glacier و mountain یا street و buildings با این حال، با توجه به دقت test برابر با 81.87%، مدل توانسته است حداقل دقت مورد نیاز سؤال را به‌دست آورد.

سوال 7)

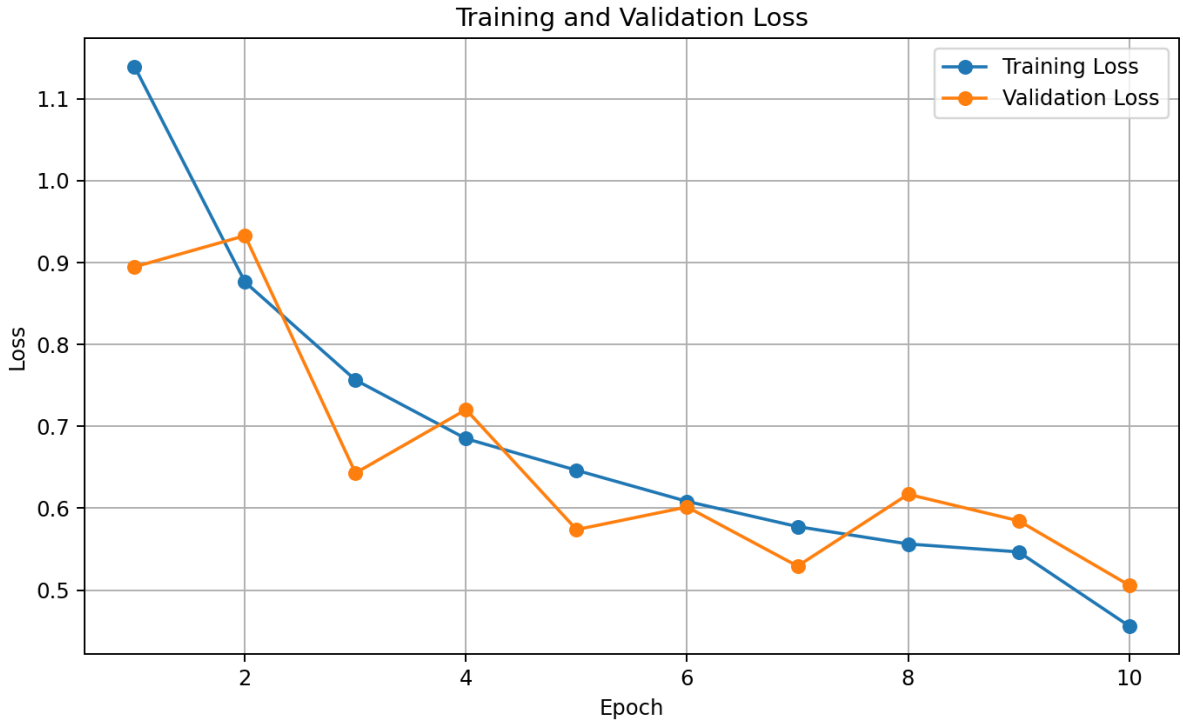
در نمودار accuracy ، روند تغییرات دقت مدل روی داده‌های آموزشی و داده‌های validation نمایش داده شده است. طبق خروجی به‌دست‌آمده، مقدار نهایی دقت روی داده‌های آموزشی برابر با 0.8337 و مقدار نهایی دقت روی داده‌های validation برابر با 0.8147 است. همچنین بهترین مقدار validation accuracy در epoch دهم به‌دست آمده و برابر با 0.8147 بوده است.

این نتیجه نشان می‌دهد که مدل در طول آموزش توانسته است ویژگی‌های مهم تصاویر را یاد بگیرد و عملکرد آن روی داده‌های validation نیز بهبود یافته است. نزدیک بودن دقت training و validation نشان می‌دهد که مدل فقط داده‌های آموزشی را حفظ نکرده، بلکه تا حد خوبی توانسته روی داده‌هایی که مستقیماً در آموزش استفاده نشده‌اند نیز عملکرد مناسبی داشته باشد.



در نمودار loss ، روند تغییرات خطای مدل روی داده‌های آموزشی و validation بررسی شده است. مقدار نهایی training loss برابر با 0.4562 و مقدار نهایی validation loss برابر با 0.5060 است. همچنین بهترین مقدار validation loss در epoch دهم به‌دست آمده و برابر با 0.5060 بوده است.

کاهش مقدار loss نشان می‌دهد که پیش‌بینی‌های مدل در طول آموزش به برچسب‌های واقعی نزدیک‌تر شده‌اند. از طرف دیگر، چون validation loss نیز تا epoch دهم به بهترین مقدار خود رسیده است، می‌توان گفت مدل تا پایان آموزش همچنان در حال بهبود بوده و افت مشخصی در عملکرد validation دیده نمی‌شود.



بررسی overfitting وunderfitting :

برای بررسی overfitting ، باید اختلاف بین عملکرد مدل روی داده‌های training و validation بررسی شود. در این خروجی، دقت training برابر با 0.8337 و دقت validation برابر با 0.8147 است. اختلاف این دو مقدار حدود 0.019 است که زیاد محسوب نمی‌شود. همچنین validation loss برابر با 0.5060و training loss برابر با 0.4562 است. اگرچه validation loss کمی بیشتر از training loss است، اما این اختلاف شدید نیست.

بنابراین، مدل نشانه‌ی overfitting شدید ندارد. اگر مدل دچار overfitting بود، انتظار می‌رفت دقت training بسیار بالا باشد اما دقت validation به‌طور واضح پایین‌تر بماند یا validation loss افزایش پیدا کند.

از طرف دیگر، مدل دچار underfitting نیز نیست؛ زیرا هم دقت training و هم دقت validation به بالای 80 درصد رسیده‌اند و loss نیز به مقدار نسبتاً مناسبی کاهش یافته است. اگر مدل underfit بود، انتظار می‌رفت هر دو دقت trainingو validation پایین باقی بمانند و مدل نتواند الگوهای اصلی داده‌ها را یاد بگیرد.

در نتیجه، با توجه به نمودارها و مقادیر نهایی، مدل توانسته است عملکرد مناسبی روی داده‌های آموزشی و validation داشته باشد و دچار overfitting یا underfitting شدید نشده است.

```
Training history columns:
Index(['epoch', 'accuracy', 'loss', 'val_accuracy', 'val_loss'], dtype='object')
Best validation accuracy:
0.8146828413009644
Best validation accuracy epoch:
10
Best validation loss:
0.506041944026947
Best validation loss epoch:
10
Final training accuracy:
0.8337192535400391
Final validation accuracy:
0.8146828413009644
Final training loss:
0.4561789035797119
Final validation loss:
0.506041944026947
Conclusion: The model does not show severe overfitting or underfitting.
```

سوال 8)

مقایسه مدل اولیه (بخش ۵) و مدل بهبودیافته (بخش ۸)

تغییرات اعمال‌شده در بخش ۸

برای بهبود عملکرد مدل اولیه، چند تغییر در معماری و فرآیند آموزش اعمال شد:

1 . اضافه شدنBatch Normalization

در مدل اولیه هیچ لایه Batch Normalization وجود نداشت، اما در مدل بهبودیافته پس از هر لایه کانولوشنی و همچنین پس از اولین لایه Dense از Batch Normalization استفاده شد.

مزایا:

- پایدارتر شدن فرآیند آموزش
- کاهش حساسیت به مقداردهی اولیه وزن‌ها
- افزایش سرعت همگرایی مدل

2 . اضافه شدن Dropout

در مدل اولیه از Dropout استفاده نشده بود.

در مدل جدید:

```
Dropout(0.2)
Dropout(0.1)
```

به شبکه اضافه شد.

مزایا:

- کاهش Overfitting
- افزایش توانایی تعمیم مدل روی داده‌های جدید

3 . افزایش تعداد لایه‌های کانولوشنی

مدل اولیه:

```
Conv2D(32)
Conv2D(64)
Conv2D(128)
Conv2D(128)
```

مدل جدید:

```
Conv2D(32)
Conv2D(64)
Conv2D(128)
Conv2D(256)
Conv2D(512)
```

بنابراین تعداد لایه‌های کانولوشنی از ۴ لایه به ۵ لایه افزایش یافت.

مزایا:

- استخراج ویژگی‌های پیچیده‌تر
- افزایش قدرت مدل در تشخیص جزئیات تصاویر

4 . افزایش تعداد فیلترهای کانولوشنی

بیشترین تعداد فیلتر در مدل اولیه 128 بود، اما در مدل جدید تا 512 فیلتر افزایش یافت.

مزایا:

- استخراج ویژگی‌های سطح بالاتر
- افزایش ظرفیت یادگیری شبکه

5 . جایگزینی Flatten با GlobalAveragePooling2D

مدل اولیه:

```
Flatten()
```

مدل جدید:

```
GlobalAveragePooling2D()
```

مزایا:

- کاهش تعداد پارامترها
- کاهش احتمال Overfitting
- حفظ مهم‌ترین ویژگی‌های استخراج‌شده

6 . بهبود بخش طبقه‌بندی

مدل اولیه:

```
Dense(128)
Dense(6)
```

مدل جدید:

```
Dense(256)
Dense(128)
Dense(6)
```

این تغییر باعث افزایش توانایی مدل در یادگیری الگوهای پیچیده‌تر شد.

7 . استفاده از EarlyStopping

در مدل جدید از EarlyStopping استفاده شد تا در صورت عدم بهبود عملکرد اعتبارسنجی، آموزش متوقف شده و بهترین وزن‌ها بازیابی شوند.

مقایسه نتایج

معیار	مدل اولیه (بخش ۵)	مدل بهبودیافته (بخش ۸)
بهترین Validation Accuracy	84.11٪	86.42٪
بهترین Validation Loss	0.439	0.370
Test Accuracy	گزارش نشده	85.87٪
Test Loss	گزارش نشده	0.391
تعداد Conv2D	4	5
Batch Normalization	ندارد	دارد
Dropout	ندارد	دارد

میزان بهبود

بهترین دقت اعتبارسنجی از:

```
84.11%
```

به:

```
86.42%
```

رسید.

بنابراین:

```
86.42 - 84.11 = 2.31%
```

مدل حدود 2.31 درصد بهبود در Validation Accuracy داشته است.

همچنین مقدار Validation Loss از:

```
0.439
```

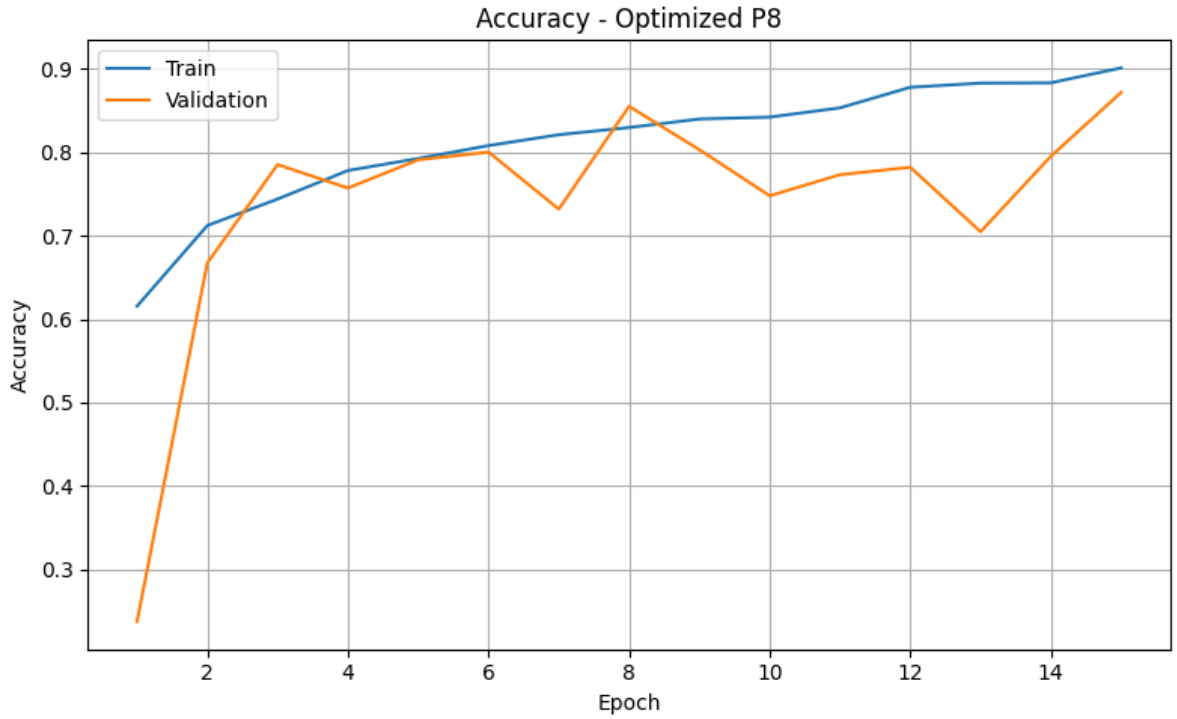
به:

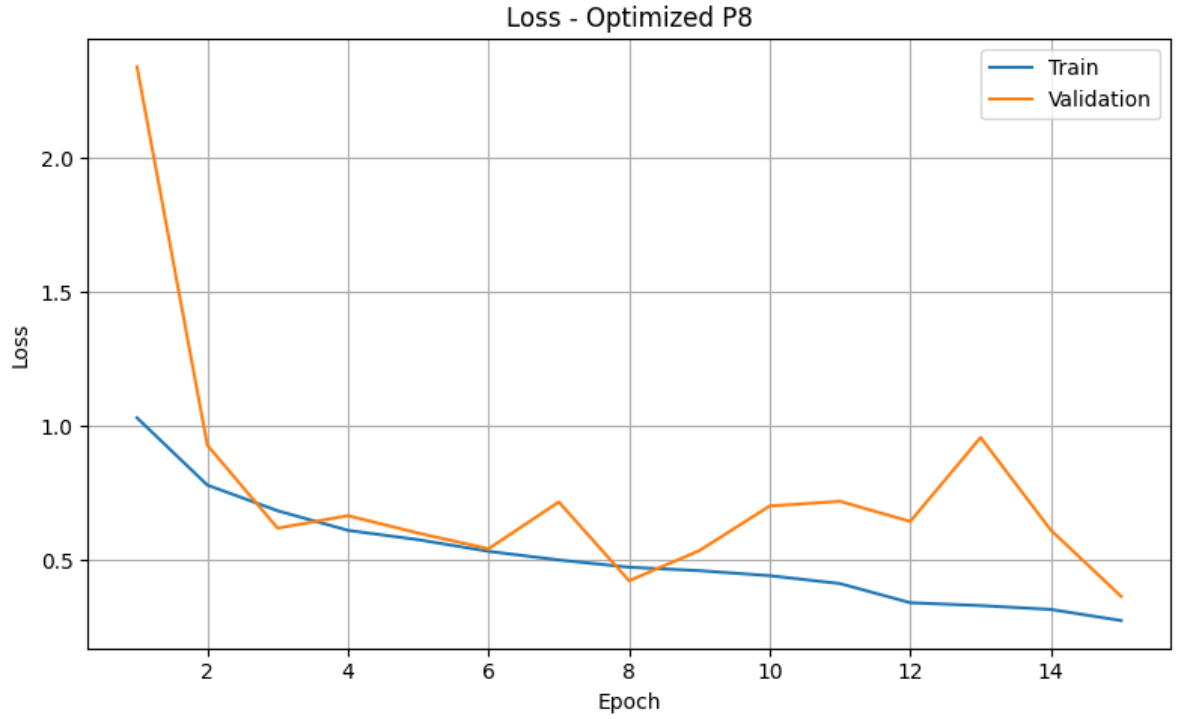
```
0.370
```

کاهش یافته است که نشان‌دهنده یادگیری بهتر و خطای کمتر مدل است.

نتیجه‌گیری

با اعمال تکنیک‌های Batch Normalization، Dropout، افزایش تعداد لایه‌ها و فیلترهای کانولوشنی، جایگزینی Flatten با GlobalAveragePooling2D و استفاده از EarlyStopping، عملکرد مدل بهبود یافت. بهترین دقت اعتبارسنجی از 84.11٪ به 86.42٪ افزایش پیدا کرد و مقدار خطا نیز کاهش یافت. همچنین مدل بهبودیافته روی داده‌های آزمون به دقت 85.87٪ دست یافت که بالاتر از حداقل دقت موردنیاز تمرین (80٪) است. این نتایج نشان می‌دهد تغییرات اعمال‌شده باعث افزایش توانایی تعمیم مدل و کاهش بیش‌برازش شده‌اند.





نمودارهای تغییرات Loss و Accuracy در طولEpoch های مختلف نشان می‌دهند که روند کلی آموزش صعودی و مناسب بوده است. با این حال، در برخیEpoch ها مانندEpoch های 9، 11 و 15 (افت ناگهانی در Validation Accuracy مشاهده می‌شود. این تغییرات شدید عمدتاً ناشی از اعمال Data Augmentation، نوسانات فرآیند بهینه‌سازی توسط الگوریتم Adam، تغییرات آماری ایجادشده توسط Batch Normalization وتغییرات لرنینگ ریت و تفاوت عملکرد مدل روی داده‌های اعتبارسنجی است. از آنجا که داده‌های آموزشی در هر Epoch به‌صورت متفاوت تغییر شکل داده می‌شوند، مدل ممکن است در برخیEpoch ها روی ویژگی‌های متفاوتی تمرکز کرده و عملکرد اعتبارسنجی به‌طور موقت کاهش یابد. با وجود این نوسانات، روند کلی نشان می‌دهد که مدل به‌تدریج به سمت همگرایی حرکت کرده و در نهایت به بهترین دقت اعتبارسنجی 86.42٪ و دقت تست 85.87٪ دست یافته است.

نتیجه ران ها:

```
Training history:
    accuracy      loss  val_accuracy  val_loss  learning_rate
0   0.541147  1.147527      0.600855  1.037336       0.000500
1   0.664767  0.887187      0.631860  0.998629       0.000500
2   0.714642  0.776881      0.720242  0.796744       0.000500
3   0.746348  0.697188      0.720955  0.772314       0.000500
4   0.770128  0.632878      0.799002  0.569833       0.000500
5   0.779391  0.609275      0.814326  0.531140       0.000500
6   0.794175  0.561109      0.797933  0.556900       0.000500
7   0.806288  0.539131      0.799715  0.567860       0.000500
8   0.835768  0.460071      0.814683  0.533914       0.000250
9   0.837905  0.448456      0.808268  0.564451       0.000250
10  0.851176  0.410080      0.806486  0.539775       0.000125
11  0.851888  0.400033      0.826800  0.489445       0.000125
12  0.863199  0.380716      0.828938  0.507614       0.000125
13  0.861774  0.379806      0.842124  0.458179       0.000125
14  0.869255  0.364467      0.846044  0.460137       0.000125
Training completed.
Best validation accuracy:
0.8460441827774048
Best epoch:
15
Training history has been saved as training_history_question_5.csv
Best model has been saved as cnn_model_best_question_5.keras
Final model has been saved as cnn_model_final_question_5.keras
(myenv) rouja@DESKTOP-FJ1VDHC:~/cv_env/CA4$
```

```
Classes: ['buildings', 'forest', 'glacier', 'mountain', 'sea', 'street']
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
sequential (Sequential)	(None, 150, 150, 3)	0
rescaling (Rescaling)	(None, 150, 150, 3)	0
conv2d (Conv2D)	(None, 150, 150, 32)	896
batch_normalization (BatchNormalization)	(None, 150, 150, 32)	128
max_pooling2d (MaxPooling2D)	(None, 75, 75, 32)	0
conv2d_1 (Conv2D)	(None, 75, 75, 64)	18,496
batch_normalization_1 (BatchNormalization)	(None, 75, 75, 64)	256
max_pooling2d_1 (MaxPooling2D)	(None, 37, 37, 64)	0
conv2d_2 (Conv2D)	(None, 37, 37, 128)	73,856
batch_normalization_2 (BatchNormalization)	(None, 37, 37, 128)	512
max_pooling2d_2 (MaxPooling2D)	(None, 18, 18, 128)	0
conv2d_3 (Conv2D)	(None, 18, 18, 256)	295,168
batch_normalization_3 (BatchNormalization)	(None, 18, 18, 256)	1,024
max_pooling2d_3 (MaxPooling2D)	(None, 9, 9, 256)	0
conv2d_4 (Conv2D)	(None, 9, 9, 512)	1,180,160
batch_normalization_4 (BatchNormalization)	(None, 9, 9, 512)	2,048
max_pooling2d_4 (MaxPooling2D)	(None, 4, 4, 512)	0
global_average_pooling2d (GlobalAveragePooling2D)	(None, 512)	0
dense (Dense)	(None, 256)	131,328
batch_normalization_5 (BatchNormalization)	(None, 256)	1,024
dropout (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 128)	32,896
dropout_1 (Dropout)	(None, 128)	0
dense_2 (Dense)	(None, 6)	774

```
 Total params: 1,738,566 (6.63 MB)
 Trainable params: 1,736,070 (6.62 MB)
 Non-trainable params: 2,496 (9.75 KB)
Epoch 1/15
2026-06-18 19:42:59.861627: I tensorflow/core/kernels/data/shuffle_dataset_op.cc:450] ShuffleDatasetV3:20: Filling up shuffle buffer (this may take a while): 244 of 2000
2026-06-18 19:43:00.240617: I tensorflow/core/kernels/data/shuffle_dataset_op.cc:480] Shuffle buffer filled.
```

```
Epoch 1/15
2026-06-18 19:42:59.861627: I tensorflow/core/kernels/data/shuffle_dataset_op.cc:450] ShuffleDatasetV3:20: Filling up shuffle buffer (this may take a while)
: 244 of 2000
2026-06-18 19:43:00.240617: I tensorflow/core/kernels/data/shuffle_dataset_op.cc:480] Shuffle buffer filled.
2026-06-18 19:43:01.823985: I external/local_xla/xla/stream_executor/cuda/cuda_dnn.cc:465] Loaded cuDNN version 8907
2026-06-18 19:43:08.633516: W external/local_tsl/tsl/framework/bfc_allocator.cc:368] Garbage collection: deallocate free memory regions (i.e., allocations)
so that we can re-allocate a larger region to avoid OOM due to memory fragmentation. If you see this message frequently, you are running near the threshold
of the available device memory and re-allocation may incur great performance overhead. You may try smaller batch sizes to observe the performance impact. Se
t TF_ENABLE_GPU_GARBAGE_COLLECTION=false if you'd like to disable this feature.

Epoch 1: val_accuracy improved from -inf to 0.23735, saving model to cnn_model_best_p8.keras
351/351 - 87s - 249ms/step - accuracy: 0.6156 - loss: 1.0319 - val_accuracy: 0.2373 - val_loss: 2.3415 - learning_rate: 5.0000e-04
Epoch 2/15

Epoch 2: val_accuracy improved from 0.23735 to 0.66750, saving model to cnn_model_best_p8.keras
351/351 - 53s - 151ms/step - accuracy: 0.7123 - loss: 0.7808 - val_accuracy: 0.6675 - val_loss: 0.9299 - learning_rate: 5.0000e-04
Epoch 3/15

Epoch 3: val_accuracy improved from 0.66750 to 0.78546, saving model to cnn_model_best_p8.keras
```

```
Epoch 3: val_accuracy improved from 0.66750 to 0.78546, saving model to cnn_model_best_p8.keras
351/351 - 52s - 149ms/step - accuracy: 0.7441 - loss: 0.6850 - val_accuracy: 0.7855 - val_loss: 0.6198 - learning_rate: 5.0000e-04
Epoch 4/15

Epoch 4: val_accuracy did not improve from 0.78546
351/351 - 51s - 146ms/step - accuracy: 0.7783 - loss: 0.6118 - val_accuracy: 0.7573 - val_loss: 0.6667 - learning_rate: 5.0000e-04
Epoch 5/15

Epoch 5: val_accuracy improved from 0.78546 to 0.79116, saving model to cnn_model_best_p8.keras
351/351 - 54s - 152ms/step - accuracy: 0.7928 - loss: 0.5769 - val_accuracy: 0.7912 - val_loss: 0.6015 - learning_rate: 5.0000e-04
Epoch 6/15

Epoch 6: val_accuracy improved from 0.79116 to 0.80043, saving model to cnn_model_best_p8.keras
351/351 - 52s - 148ms/step - accuracy: 0.8082 - loss: 0.5335 - val_accuracy: 0.8004 - val_loss: 0.5429 - learning_rate: 5.0000e-04
Epoch 7/15
2026-06-18 19:48:28.885267: W tensorflow/core/kernels/data/prefetch_autotuner.cc:52] Prefetch autotuner tried to allocate 8640256 bytes after encountering t
he first element of size 8640256 bytes.This already causes the autotune ram budget to be exceeded. To stay within the ram budget, either increase the ram bu
dget or reduce element size

Epoch 7: val_accuracy did not improve from 0.80043
351/351 - 53s - 152ms/step - accuracy: 0.8213 - loss: 0.5013 - val_accuracy: 0.7320 - val_loss: 0.7177 - learning_rate: 5.0000e-04
Epoch 8/15
2026-06-18 19:49:21.985344: W tensorflow/core/kernels/data/prefetch_autotuner.cc:52] Prefetch autotuner tried to allocate 8640256 bytes after encountering t
he first element of size 8640256 bytes.This already causes the autotune ram budget to be exceeded. To stay within the ram budget, either increase the ram bu
dget or reduce element size

Epoch 8: val_accuracy improved from 0.80043 to 0.85531, saving model to cnn_model_best_p8.keras
351/351 - 52s - 148ms/step - accuracy: 0.8299 - loss: 0.4741 - val_accuracy: 0.8553 - val_loss: 0.4236 - learning_rate: 5.0000e-04
Epoch 9/15
2026-06-18 19:50:13.812344: W tensorflow/core/kernels/data/prefetch_autotuner.cc:52] Prefetch autotuner tried to allocate 8640256 bytes after encountering t
he first element of size 8640256 bytes.This already causes the autotune ram budget to be exceeded. To stay within the ram budget, either increase the ram bu
dget or reduce element size

Epoch 9: val_accuracy did not improve from 0.85531
351/351 - 53s - 151ms/step - accuracy: 0.8401 - loss: 0.4613 - val_accuracy: 0.8033 - val_loss: 0.5363 - learning_rate: 5.0000e-04
Epoch 10/15
2026-06-18 19:51:06.807013: W tensorflow/core/kernels/data/prefetch_autotuner.cc:52] Prefetch autotuner tried to allocate 8640256 bytes after encountering t
he first element of size 8640256 bytes.This already causes the autotune ram budget to be exceeded. To stay within the ram budget, either increase the ram bu
dget or reduce element size

Epoch 10: val_accuracy did not improve from 0.85531
351/351 - 53s - 151ms/step - accuracy: 0.8424 - loss: 0.4426 - val_accuracy: 0.7480 - val_loss: 0.7028 - learning_rate: 5.0000e-04
Epoch 11/15
2026-06-18 19:51:59.957484: W tensorflow/core/kernels/data/prefetch_autotuner.cc:52] Prefetch autotuner tried to allocate 8640256 bytes after encountering t
he first element of size 8640256 bytes.This already causes the autotune ram budget to be exceeded. To stay within the ram budget, either increase the ram bu
dget or reduce element size
```

```
Epoch 11: val_accuracy did not improve from 0.85531

Epoch 11: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.
351/351 - 53s - 150ms/step - accuracy: 0.8535 - loss: 0.4131 - val_accuracy: 0.7733 - val_loss: 0.7201 - learning_rate: 5.0000e-04
Epoch 12/15
2026-06-18 19:52:52.665372: W tensorflow/core/kernels/data/prefetch_autotuner.cc:52] Prefetch autotuner tried to allocate 8640256 bytes after encountering t
he first element of size 8640256 bytes.This already causes the autotune ram budget to be exceeded. To stay within the ram budget, either increase the ram bu
dget or reduce element size

Epoch 12: val_accuracy did not improve from 0.85531
351/351 - 53s - 151ms/step - accuracy: 0.8782 - loss: 0.3417 - val_accuracy: 0.7823 - val_loss: 0.6450 - learning_rate: 2.5000e-04
Epoch 13/15
2026-06-18 19:53:45.823829: W tensorflow/core/kernels/data/prefetch_autotuner.cc:52] Prefetch autotuner tried to allocate 8640256 bytes after encountering t
he first element of size 8640256 bytes.This already causes the autotune ram budget to be exceeded. To stay within the ram budget, either increase the ram bu
dget or reduce element size

Epoch 13: val_accuracy did not improve from 0.85531
351/351 - 53s - 152ms/step - accuracy: 0.8831 - loss: 0.3310 - val_accuracy: 0.7049 - val_loss: 0.9585 - learning_rate: 2.5000e-04
Epoch 14/15
2026-06-18 19:54:38.992603: W tensorflow/core/kernels/data/prefetch_autotuner.cc:52] Prefetch autotuner tried to allocate 8640256 bytes after encountering t
he first element of size 8640256 bytes.This already causes the autotune ram budget to be exceeded. To stay within the ram budget, either increase the ram bu
dget or reduce element size

Epoch 14: val_accuracy did not improve from 0.85531

Epoch 14: ReduceLROnPlateau reducing learning rate to 0.0001250000059371814.
351/351 - 54s - 153ms/step - accuracy: 0.8835 - loss: 0.3166 - val_accuracy: 0.7954 - val_loss: 0.6123 - learning_rate: 2.5000e-04
Epoch 15/15
2026-06-18 19:55:32.721518: W tensorflow/core/kernels/data/prefetch_autotuner.cc:52] Prefetch autotuner tried to allocate 8640256 bytes after encountering t
he first element of size 8640256 bytes.This already causes the autotune ram budget to be exceeded. To stay within the ram budget, either increase the ram bu
dget or reduce element size

Epoch 15: val_accuracy improved from 0.85531 to 0.87206, saving model to cnn_model_best_p8.keras
351/351 - 54s - 154ms/step - accuracy: 0.9014 - loss: 0.2752 - val_accuracy: 0.8721 - val_loss: 0.3653 - learning_rate: 1.2500e-04
Restoring model weights from the end of the best epoch: 15.
94/94 ━━━━━━━━━━━━━━━━━━━━ 6s 47ms/step - accuracy: 0.8883 - loss: 0.3217

=== TEST RESULTS ===
Test loss: 0.3832884132862091
Test accuracy: 0.8663333058357239

Best val accuracy: 0.8720598816871643
Best val loss: 0.36529335379600525
```

نکته: بخش 5 یکبار با epoch 15 ران شد چون برای بهبود مدل در epoch های پایینتر مشاهده تاثیر ممکن نبود، پس باید هر دو مدل را به 15 می رساندیم که مقایسه ممکن شود.